

Unit 5.3



Transaction:

- Concept
- Properties
- States of Transaction



Prepared By
Mr. Mohammad Ali
Lecturer (Selection Grade)
M. H. Saboo Siddik Polytechnic, Mumbai



Transaction - Concept

■ Single User VS Multi User Systems

- One criteria to classify Database is according to number of user that concurrently connect to the system.

- **Single-User System:**

- At most one user at a time can use the system.

- **Multiuser System:**

- Many users can access the system concurrently(same time).

■ Concurrency

- **Interleaved processing:**

- Concurrent execution of processes is interleaved in a single CPU

- **Parallel processing:**

- Processes are concurrently executed in multiple CPUs.



Transaction - Concept

■ What is a Transaction?

- Transaction is an **executing program** that forms a **logical unit of database processing** that includes one or more access operations (read –retrieval or access, write - insert or update, delete).
- A **transaction** (set of operations) may be stand-alone specified in a high level language like SQL submitted interactively, or may be embedded within a program.
- **Transaction boundaries:**
 - Begin and End transaction.
- An **application program** may contain several transactions separated by the Begin and End transaction boundaries.
- If the database operations in a transaction do not update the database, it is called “**Read-only transaction**”

Example of Transaction

Transaction T1

- Let T_1 be a transaction that transfer money from account A (5000) to account B. The transaction can be defined as

read (A)	(withdraw from A)
A := A - 5000	
write (A);	(update A)
read (B)	(deposit to B)
B := B + 5000	
write (B);	(update B)

Transaction T2

- Let T2 be a transaction that transfer 10% of account A to Account B. The transaction can be defined as

Read (A)	(withdraw from A)
temp := 0.1*A	
A:= A-temp	
Write (A)	(update A)
Read (B)	(deposit to B)
B := B + temp	
Write (B)	(update B)



Transaction - Concept

- Basic unit of data transfer from the disk to the computer main memory is one block. In general, a data item (what is read or written) will be the field of some record in the database, although it may be a larger unit such as a record or even a whole block.
- Basic operations are **read** and **write**
 - **read_item(X)**: Reads a database item named X into a program variable X.
 - **write_item(X)**: Writes the value of program variable X into the database item named X.



Transaction - Concept

READ OPERATION:

- **read_item(X)** command includes the following steps:
 - Find the address of the disk block that contains item X.
 - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 - Copy item X from the buffer to the program variable named X.



Transaction - Concept

WRITE OPERATION:

- **write_item(X)** command includes the following steps:
 - Find the address of the disk block that contains item X.
 - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 - Copy item X from the program variable named X into its correct location in the buffer.
 - Store the updated block from the buffer back to disk (either immediately or at some later point in time).



Properties of Transaction : **ACID** Properties

- To ensure integrity of data, we require that the database system maintain the following properties of the transactions:
 - **Atomicity.**
 - **Consistency preservation.**
 - **Isolation.**
 - **Durability or permanency.**

Atomicity

- **Atomicity:** A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.
- Either all operations of the transaction are reflected properly in the database, or none are. (It is the responsibility of recovery)
 - State before the execution of transaction T_i
 - The value of A = 50,000
 - The value of B = 100
 - Failure occur (ex. Hardware failure)
 - Failure happen after the **WRITE(A)** operation
 - (at this moment A = 50000 – 5000 = 45000)
 - And the value of B = 100 (inconsistency state)
 - In consistency state A = 45000 and B = 5100



Atomicity(cont.)

- Idea behind ensuring atomicity is following:
 - **The database system keeps track of the old values of any data on which a transaction performs a write**
 - **If the transaction does not complete, the DBMS restores the old values to make it appear as though the transaction have never execute.**



Consistency

- **Consistency preservation: A correct execution of the transaction must take the database from one consistent state to another.**
- The consistency requirement here is that the **sum of A and B** be unchanged by the execution of the transaction.
- Without consistency requirement, money could be created or destroyed by the transaction.
- It can be verified,
 - **If the database is in state of consistency before an execution of the transaction, the database remains consistent after the execution of the transaction.**
 - (It is the responsibility of the applications and DBMS to maintain the constraints)



Isolation

- **Isolation:** the execution of the transaction should be isolated from other transactions (Locking - It is the responsibility of concurrency)
- **A transaction should not make its updates visible to other transactions until it is committed; this property, when enforced strictly, solves the temporary update problem and makes cascading rollbacks of transactions unnecessary**
- **Isolation.** Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j , it appears to T_i that
 - either T_j finished execution before T_i started,
 - or T_j started execution after T_i finished.
 - Thus, each transaction is unaware of other transactions executing concurrently in the system.

(Execution of transaction should not be interfered with by any other transactions executing concurrently)



Isolation

- **Isolation** : * Isolation level:
 - Level 0 (no dirty read)
 - Level 1 (no lost update)
 - Level 2 (no dirty+ no lost)
 - Level 3 (level 2+repeatable reads)



Durability or permanency

- **Durability or permanency.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.
- **These changes must not be lost because of any failure**
- ensures that, transaction has been committed, that transaction's updates do not get lost, even if there is a system failure
- **(It is the responsibility of recovery)**



States of Transaction

- A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all.
 - For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits or aborts.
- **Transaction states:**
 - Active state
 - Partially committed state
 - Committed state
 - Failed state
 - Terminated State



States of Transaction

- Recovery manager keeps track of the following operations:
 - **begin_transaction**: This marks the beginning of transaction execution.
 - **read** or **write**: These specify read or write operations on the database items that are executed as part of a transaction.
 - **end_transaction**: This specifies that read and write transaction operations have ended and marks the end limit of transaction execution.
 - At this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database or whether the transaction has to be aborted because it violates concurrency control or for some other reason.



States of Transaction

- Recovery manager keeps track of the following operations (cont):
 - **commit_transaction**: This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
 - **rollback** (or **abort**): This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.



States of Transaction

- Recovery techniques use the following operators:
 - **undo**: Similar to rollback except that it applies to a single operation rather than to a whole transaction.
 - **redo**: This specifies that certain *transaction operations* must be *redone* to ensure that all the operations of a committed transaction have been applied successfully to the database.

States of Transaction

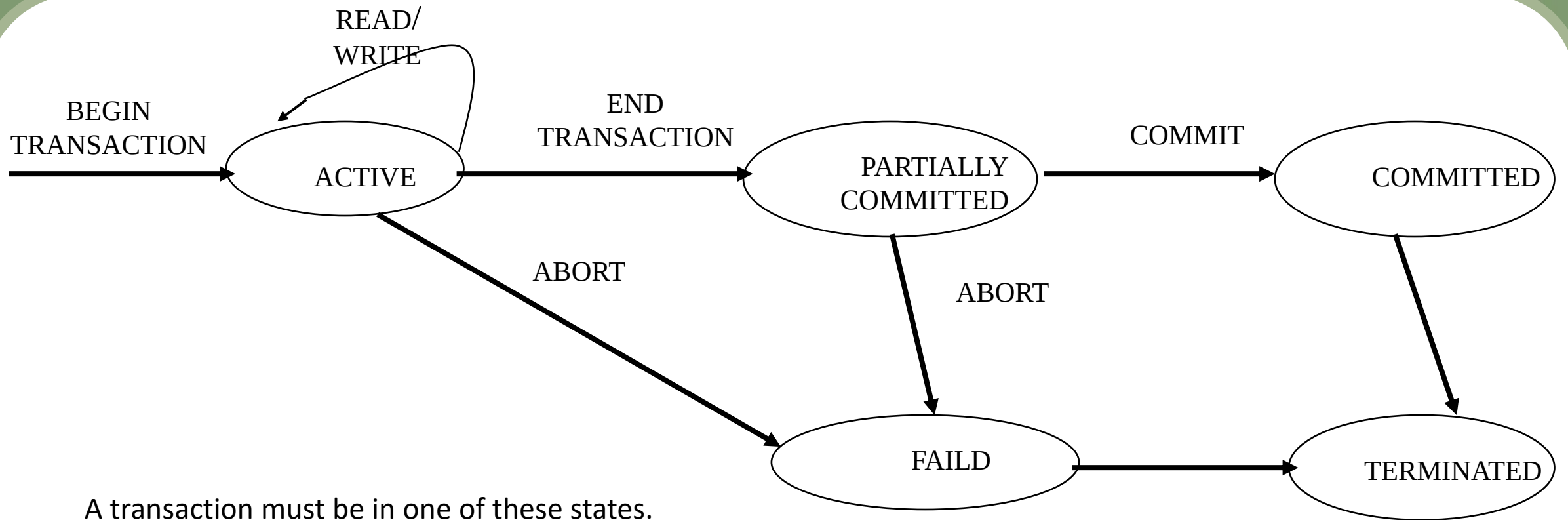


Figure State transition diagram illustrating the states for transaction execution

Begin, Active, and Commit

- begin transaction : the first SQL query at the beginning of a program or a user session, or after the commitment of an earlier transaction
- Active : running SQL queries & updates. Operate on data in buffer or on disk
 - **Active**, the initial state; the transaction stays in this state while it is executing.
- Partially-committed : just before “commit work” is executed. All operations of the transaction have been done (in the memory buffer or on disk) but not yet final.
 - **Partially committed**, after the final statement has been executed
- Committed : after “commit work” is executed. Data updates have been made permanent. **Committed**, after successful completion



Failed and Aborted

- Failed : **Failed**, after the discovery that normal execution can no longer proceed.
- after an exception is encountered. Normal execution is interrupted. Can be
 - Programmed
 - DBMS enforced (e.g., for concurrency control)
 - System failure: power failure, disk crashes,
- Aborted : after “rollback work” is executed. All actions of the transaction have been undone and the database has been restored to its state prior to the start of the transaction.



Committed and Terminated

- The transaction has **committed** only if it has entered the **committed state**.
- The transaction has **aborted** only if it has entered the **aborted state**.
- The transaction is said to have **terminated** if has either **committed or aborted**.